BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Nesnelerin İnşası ve Temel Mekanikler

EBRU AYDOĞAN

Neden Sınıflara (Class) İhtiyacımız Var?

- Fonksiyonel programlamada veriyi ve işlemi birbirinden ayrı tutuyorduk (veriyi fonksiyona gönderiyorduk).
- Ancak gerçek dünya projelerinde (örneğin bir e-ticaret sitesinde) veri ile o verinin yapabileceği işlemleri "paketlemek" işimizi kolaylaştırır.

Neden Sınıflara (Class) İhtiyacımız Var?

- Analoji (Mimari Çizim):
  - **Class (Sınıf):** Bir evin mimari çizimidir. Hangi odalar olacak, duvarlar nereye örülecek bellidir ama içinde yaşayamazsınız.
  - **Object (Nesne):** O çizime göre inşa edilmiş gerçek bir evdir. Artık içine eşya koyabilir, boyayabilir veya oturabilirsiniz. Aynı çizimden (Class) binlerce farklı ev (Object) inşa edebiliriz.

Temel Sınıf Tanımlama

- Python'da bir sınıf tanımlamak için class anahtar kelimesini kullanırız.(Pek çok programlama dilinde olduğu gibi.)
- Henüz bir "yapıcı metot" (constructor) kullanmadan, nesnelerimizi nasıl oluşturduğumuza ve onlara veri atadığımıza bakalım.

Temel Sınıf Tanımlama

```python
class Product:
    pass

# 2. Nesneyi (Object) oluşturuyoruz (Mimariye göre inşa etme)
p1 = Product()

# 3. Nesneye manuel olarak özelliklerini atıyoruz
p1.name = "Samsung S24"
p1.price = 40000

# 4. Başka bir nesne oluşturuyoruz
p2 = Product()
p2.name = "iPhone 15"
p2.price = 35000

# Özellikleri kullanıyoruz
print(f"Ürün 1: {p1.name}, Fiyat: {p1.price}")
print(f"Ürün 2: {p2.name}, Fiyat: {p2.price}")
```

BTK AKADEMİ

Temel Sınıf Tanımlama

- Bu Yaklaşımın "Eksik" Yanları
  - Bu örnekte gördüğümüz gibi, Product sınıfından her yeni nesne ürettiğimizde, name ve price değişkenlerini **manuel olarak** dışarıdan tanımlamak zorundayız.
    - **Risk:** Eğer bir nesne oluştururken name tanımlamayı unutursak ne olur?
    - **Verimlilik:** 100 tane ürün oluşturmamız gerekse, 100 kere p.name ve p.price yazmak zorunda kalırız.
  - İşte tam bu noktada, nesne oluşturulduğu anda çalışan ve "Senin adın ne, fiyatın ne?" diye otomatik soru soran bir mekanizmaya ihtiyaç duyarız. Buna **Constructor** (Yapıcı Metot) diyoruz.

Neden Constructor (__init__) Kullanıyoruz?

- Daha önce de belirttiğimiz gibi, nesne oluşturduktan sonra p1.name = ... diye manuel atama yapmak hem zahmetli hem de hata yapmaya açık bir süreçtir.
- Eğer bir özelliği tanımlamayı unutursak ne olur? Kodumuz çalışma anında patlar (hata verir).

BTK AKADEMİ

Neden Constructor (__init__) Kullanıyoruz?

- **Constructor (Yapıcı Metot),** nesne oluşturulduğu anda "Bu nesnenin temel özellikleri ne olmalı?" sorusunu soran ve bu verileri nesnenin hafızasına (belleğine) otomatik olarak kazıyan özel bir metottur.
- **Örnek:** Bir okul kayıt sistemi düşünün. Öğrenci sınıfa girer girmez (nesne oluşturulduğunda) ona hemen isim, numara ve bölüm etiketi yapıştırılıyor. Bunu elle yapmıyoruz, kayıt sistemi (Constructor) bunu otomatik yapıyor.

«self» Parametresi: "Bu Nesne Benim"

- Neden self yazıyoruz?
  - self, oluşturulan **nesnenin kendisini** temsil eder.
  - Python, Product() sınıfından binlerce nesne üretebilir.
  - self anahtar kelimesi, fonksiyonun hangi nesnenin verisiyle işlem yapacağını şaşırmamasını sağlar.

BTK AKADEMİ

Manuel Tanımlamadan Constructor Yapısına

- Şimdi önceki manuel örneğimizi, profesyonel (constructor içeren) yapıya çevirelim.
- Önceki Manuel Yapı:

```
# Hatalı/Zahmetli Yapı:
class Product:
    pass

p1 = Product()
p1.name = "Samsung S24" # Her seferinde yazmak zorunda kalıyoruz
```

BTK AKADEMİ

Manuel Tanımlamadan Constructor Yapısına

- Constructor ile Dönüştürülmüş Yapı:

```
class Product:
    # Constructor: Nesne doğduğu an burası çalışır
    def __init__(self, name, price):
        # 'self' ile gelen bilgiyi nesnenin içine gömüyoruz
        self.name = name
        self.price = price

# Artık nesne oluştururken veriyi en başta veriyoruz (Daha temiz!)
p1 = Product("Samsung S24", 40000)
p2 = Product("iPhone 15", 35000)

print(f"Ürün 1: {p1.name}, Fiyat: {p1.price}")
print(f"Ürün 2: {p2.name}, Fiyat: {p2.price}")
```

BTK AKADEMİ

Manuel Tanımlamadan Constructor Yapısına

- Kritik Noktalar:
  - Otomatik Çalışma: __init__ metodunu biz p1.init() diye çağırmıyoruz. Python bunu Product(...) dendiği an otomatik olarak çağırıyor.
  - Bellek Kapsamı: self.name = name satırındaki self.name nesnenin hafızasında kalıcı olan özelliktir. Sağ taraftaki name ise sadece o an fonksiyona gelen geçici veridir.
  - self Yazılmazsa: Eğer self yazmazsak, Python o verinin hangi nesneye ait olduğunu bilemez ve hata verir.

Instance Methods (Nesne Metotları)

- Fonksiyondan Metoda Geçiş
  - Geçen hafta boyunca dışarıda tanımladığımız ve veriyi parametre olarak gönderdiğimiz fonksiyonlar yazdık.
  - Şimdi ise şunu söylüyoruz: **"Veriyi zaten nesnenin içinde tutuyoruz (self ile), o zaman işlemi de o verinin hemen yanına, yani sınıfın içine taşıyalım."**
  - Bir sınıfın içine yazdığımız ve self parametresini alan her fonksiyona **Instance Method (Nesne Metodu)** diyoruz.
  - Bu metotlar, "o nesnenin yapabileceği işleri" temsil eder.

Instance Methods (Nesne Metotları)

- Nasıl Tanımlanır ve Kullanılır?
  - Örnek: Bir Person (Kişi) sınıfı üzerinden gidelim. Bu sınıfın introduce (kendini tanıtma) diye bir metodu olsun.

```
class Person:
    # Constructor: Nesnenin verilerini oluşturduk
    def __init__(self, name, year):
        self.name = name
        self.year = year

    # Instance Method: Nesnenin yapabileceği bir iş
    def introduce(self):
        # 'self' sayesinde, şu an hangi nesne (p1 mi p2 mi?)
        # çağrılıyorsa onun verisine ulaşıyoruz.
        return f"Benim adım {self.name} ve doğum yılım {self.year}."
```

BTK AKADEMİ

Instance Methods (Nesne Metotları)

- Nasıl Tanımlanır ve Kullanılır?
  - Örnek: Bir Person (Kişi) sınıfı üzerinden gidelim. Bu sınıfın introduce (kendini tanıtma) diye bir metodu olsun ve kullanalım.

```
# Nesneleri oluşturuyoruz
p1 = Person("Ali", 1990)
p2 = Person("Ayşe", 1995)

# Metodu çağırıyoruz
# Buradaki p1.introduce() çağrısı, aslında Python'a
# "p1'i al ve introduce metoduna self olarak gönder" demektir.
print(p1.introduce())
print(p2.introduce())
```

BTK AKADEMİ

Class Attributes & Class Methods

- Sınıfın Paylaşılan Hafızası
  - Şimdiye kadar yaptığımız tüm örneklerde, self.name, self.year gibi **Instance Attributes** kullandık. Bunlar "her nesnenin kendine ait" verileriydi (her insanın adı farklıdır).
  - Peki ya **"Tüm nesneler için ortak olan"** bir veri tutmamız gerekirse? Örneğin, bir oyunda oluşturulan toplam karakter sayısı veya bir sınıftaki tüm öğrencilerin ortak okul ismi.
  - İşte burada **Class Attributes** devreye girer.

Class Attributes (Sınıf Nitelikleri)

- Class Attribute'lar, sınıfın içinde ama hiçbir metodun (constructor dahil) *içinde değil*, en tepede tanımlanır. Bunlar "kalıbın" kendisine aittir, nesneye değil.
- Örnek: Person sınıfımızdan kaç kişi oluşturulduğunu sayan bir sayaç ekleyelim.

```
class Person:
    # Class Attribute (Paylaşılan alan)
    # Bu değişken tüm Person nesneleri tarafından ortak kullanılır
    count = 0

    def __init__(self, name):
        self.name = name # Instance Attribute
        # Sayaç her nesne oluşturulduğunda sınıf üzerinden artar
        Person.count += 1
```

BTK AKADEMİ

Class Attributes (Sınıf Nitelikleri)

- Örnek: Person sınıfımızdan kaç kişi oluşturulduğunu sayan bir sayaç ekleyelim.

```
p1 = Person("Ali")
p2 = Person("Ayşe")

print(f"Toplam kişi sayısı: {Person.count}") # 2
```

- **Önemli:** Neden self.count yazmadık? Çünkü self dersek her nesnenin kendi içinde ayrı bir count değişkeni olurdu. Person.count dediğimizde, hepsi aynı "kutuya" (sınıfın ortak hafızasına) erişir.

Class Methods (@classmethod)

- Instance metotları (Bölüm 3) nasıl ki nesnenin verisini (self) manipüle ediyorsa, **Class Methods** da sınıfın verisini (cls) manipüle eder.
- **İpucu:** Bir metodun başında @classmethod yazıyorsa, o metot nesneye değil, sınıfın kendisine hizmet eder. Parametre olarak self almaz, bunun yerine **cls** (class kısaltması) alır.

Class Methods (@classmethod)

```python
class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

    # Class Method: Sınıfın verisine (count) erişir
    @classmethod
    def get_count(cls):
        return f"Şu ana kadar {cls.count} kişi oluşturuldu."

# Nesne oluşturmadan bile çağırabiliriz!
print(Person.get_count()) # "Şu ana kadar 0 kişi oluşturuldu."
```

BTK AKADEMİ

Class Methods (@classmethod)

- Kritik Notlar
  - self: "Benim (nesnenin) verilerimle uğraşıyorum."
  - cls: "Sınıfın (şablonun) ortak verileriyle uğraşıyorum."
  - **Kullanım Amacı**: Class Method'ları genellikle bir nesne oluşturmadan (örneğin veritabanından veri çekip sınıfı başlatmak için) veya paylaşılan verileri (sayaçlar, konfigürasyonlar) yönetmek için kullanırız.
  - **Erişim:** Person.count diyerek sınıfın değişkenine dışarıdan da erişilebilir (buna "public attribute" denir), ilerleyen bölümlerde bunu nasıl "korumalı" (private) hale getireceğimizi (_ veya __ ile) konuşacağız.

Uygulama 1: Öğrenci Durum Takibi

- **Görev:** Bir Ogrenci sınıfı oluşturun.
- **Gereksinimler:**
  - __init__ metodu ile öğrencinin ad, soyad ve ders_notu değerlerini alın.
  - durum_sorgula() adında bir instance metodu yazın. Bu metot, öğrencinin notu 50 ve üzerindeyse "Başarılı", değilse "Başarısız" döndürsün.
  - 2 farklı öğrenci nesnesi oluşturup durum_sorgula() metodunu test edin.

Uygulama 2: Ürün Sayacı

- **Görev:** Bir Urun sınıfı oluşturun.
- **Gereksinimler:**
  - Sınıf seviyesinde urun_adet adında bir değişken tanımlayın (başlangıç değeri 0).
  - __init__ içinde her yeni ürün oluşturulduğunda urun_adet değişkenini 1 artırın.
  - @classmethod olarak toplam_urun_sayisini_getir() adında bir metot yazın ve bu metot mevcut ürün sayısını ekrana yazdırsın.

BTK AKADEMİ

Uygulama 3: Robot Dünyası

- **Görev:** Bir Robot sınıfı oluşturun.
- **Gereksinimler:**
  - Her robotun bir isim özelliği olsun (__init__ ile alınacak).
  - Sınıf seviyesinde toplam_robot_sayisi değişkeni olsun ve her yeni robotta artsın.
  - selamla() adında bir instance metodu yazın; bu metot "Merhaba, ben [isim]" desin.
  - @classmethod ile robot_sayisini_sorgula() adında bir metot yazın; bu metot "Şu an dünyada X tane robot var" şeklinde cevap versin.

BTK AKADEMİ